



Welke beveiligingsproblemen komen voor bij REST-API's en hun authenticatiesystemen, en hoe kunnen deze worden voorkomen?

2025 – 2026
GDC – 3^{De} Graad

Ian-Chains Baute – 6ICWE
Generieke Doorstroom Competentie
Informaticawetenschappen

GDC Mentor: J. Gillis

Woord vooraf

Digitale systemen en softwarepakketten worden steeds groter, uitgebreider en complexer maar de veiligheid en beveiliging mag niet vergeten worden, deze beveiliging wordt ook steeds uitdagender en complexer samen met de groei van de software. Uitgebreide software systemen maken vaak gebruik van API's om data uit te wisselen tussen de gebruiker of andere software systemen en daarbij is de REST-API de meest voorkomende en gebruikte.

Ik heb zelf al meerdere keren REST-API's gebruikt in projecten voor het uitwisselen van data tussen de gebruiker en de app. Maar ik heb zelf nog nooit echt stil gestaan bij de beveiliging van de API voor mijn projecten. Daarom wil ik in dit onderzoek dan ook de nadruk leggen op de beveiliging van een REST-API systeem om bij te leren over welke beveiligingsproblemen er vaak aanwezig zijn en hoe ze voorkomen kunnen worden.

Dit onderzoek over de beveiliging van REST-API's heb ik uitgevoerd als voorbereiding op een vervolg project waarbij ik deze informatie over de beveiliging kan gebruiken om op een veilige manier een API systeem op te zetten bij mijn vervolg projecten.

Graag wil ik een aantal mensen vermelden en banken voor hun assistentie bij dit GDC onderzoek. Ik wil in het bijzonder mijn mentor meneer Gillis bedanken voor de steun en tips tijdens dit onderzoek.

Inhoudsopgave

Woord vooraf.....	1
Inleiding.....	3
Afkortingen.....	4
1 XSS - Cross-Site Scripting	5
1.1 Wat is XSS?.....	5
1.1.1 Algemeen.....	5
1.1.2 Reflected XSS.....	6
1.1.3 Stored XSS	6
1.2 Hoe XSS voorkomen?.....	7
1.2.1 Algemeen.....	7
1.2.2 HTML escaping - HTML entity encoding.....	7
1.2.3 HTML Sanitization & HTML Stripping	7
1.2.4 Cookies met HttpOnly attribuut.....	8
1.3 Praktisch.....	9
1.3.1 Reflected XSS bij onveilige API.....	9
1.3.2 Stored XSS bij onveilige API	10
1.3.3 Veilige API, beschermt tegen XSS	12
2 CSRF - Cross-Site Request Forgery	13
2.1 Wat is CSRF?	13
2.1.1 Algemeen.....	13
2.1.2 Voorbeeld Normaal Request	13
2.1.3 Voorbeeld CSRF	14
2.2 Hoe CSRF voorkomen?	15
2.2.1 SOP & CORS.....	15
2.2.2 Server Side Origin Check.....	15
2.2.3 CSRF Tokens	15
2.3 Praktisch.....	15
Besluit	16
Bijlagen	17
Referentielijst	17

Inleiding

In dit werk wordt er een technisch onderzoek gedaan naar API¹ systemen zowel theoretisch als praktisch, om de algemene beveiliging van API systemen beter te begrijpen en hierop te anticiperen. Dit onderzoek gaat dieper in op 2 veelvoorkomende beveiligingsproblemen bij REST-API's² met de focus op het uitwisselen van data tussen een gebruiker en een applicatie.

De 2 beveiligingsproblemen die onderzocht worden zijn: Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF) en improper input validation. Het 1^{ste} deel van elk beveiligingsprobleem bestaat uit een theoretisch onderzoek naar de aard van het probleem, uitgevoerd door een literatuurstudie om beter te begrijpen wat het probleem precies inhoudt.

Het 2^{de} deel van het onderzoek bestaat uit een praktisch productie en ontwikkelingsgedeelte, hierbij worden 2 overkoepelende API systemen opgezet. Het doel en de functionaliteiten van de 2 API systemen is hetzelfde, namelijk het kunnen aanmaken en inloggen van een account en daarnaast het aanmaken en opvragen van (blog)posts.

De 2 API systemen verschillen in de vorm van onveilig en veilig, waarbij er bij het 1^{ste} systeem geen extra acties en stappen ondernomen worden voor het voorkomen van de onderzochte beveiligingsproblemen. Bij het 2^{de} systeem zijn deze extra acties en stappen wel ondernomen. Door beide systemen met elkaar te vergelijken en te kijken naar de extra acties die ondernomen worden voor het veilig maken van het API systeem, kan worden aangetoond welke maatregelen nodig zijn om de geïdentificeerde beveiligingsproblemen te voorkomen.

Het praktische gedeelte wordt volledig gedocumenteerd via een GitHub repository, de link naar de repository is opgenomen in de bijlagen van dit werk. Bij het praktische gedeelte is er gebruik gemaakt van Python en FastAPI om een API systeem op te zetten, meer informatie over de gebruikte technologiestack kan terug gevonden worden in de repository.

¹ Informatie over wat een API is en hoe het technisch werkt kan gevonden worden in dit artikel van Geeks For Geeks: GeeksforGeeks. (2025, December 15). *What is an API (Application Programming Interface)*. GeeksforGeeks. <https://www.geeksforgeeks.org/software-testing/what-is-an-api/>

² Informatie over wat een REST-API is en hoe het technisch werkt kan gevonden worden in dit artikel van Postman: Team, P. (2025, December 16). *What is a REST API? Examples, uses, and challenges*. Postman Blog. <https://blog.postman.com/rest-api-examples/>

Afkorting

GDC	Generieke Doorstroom Competentie
i.p.v.	in plaats van
i.v.m.	in verband met
API	Application Programming Interface
REST	Representational State Transfer
XSS	Cross-Site Scripting
CSRF	Cross-Site Request Forgery
SOP	Same Origin Policy
CORS	Cross-Origin Resource Sharing
DOM	Document Object Model
DB	Database
Py	Python
JS	JavaScript
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
URL	Uniform Resource Locator
webapp	webapplicatie
UID	Unique Identifier

1 XSS - Cross-Site Scripting

1.1 Wat is XSS?

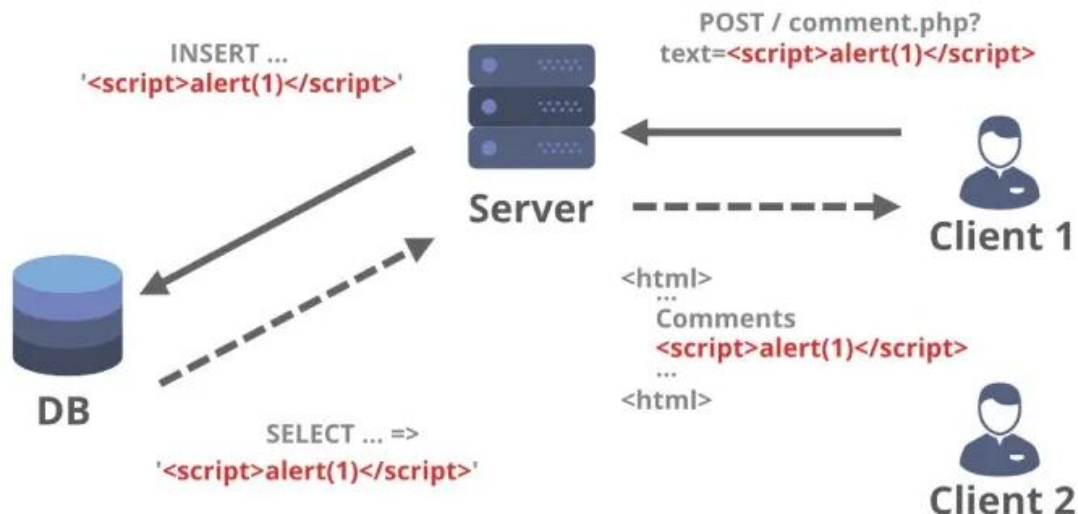
1.1.1 Algemeen

XSS is een kwetsbaarheid in een webapp dat iemand toelaat om JavaScript te injecteren in de webapp en die te laten uitvoeren in de browser van een andere gebruiker in naam van de vertrouwde maar kwetsbare webapplicatie. Daarbij draait dus de JavaScript code in de browser van het slachtoffer, wat een gewone gebruiker of een beheerder kan zijn, en de browser denkt daarbij dat de code afkomstig is van de vertrouwde webapp. Hierdoor kan de code gevoelige zaken uitvoeren zoals:

- (cookies met) sessie gegevens stelen
- zich voordoen als een andere gebruiker
- formulieren manipuleren
- Ongewenste API requests versturen
- content van de website aanpassen
- ...

Figuur 1

Visualisatie (Stored) XSS



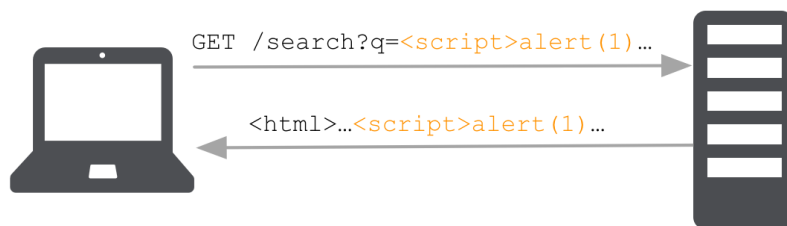
GeeksforGeeks (2025)

Figuur 1 is een visuele weergave van een XSS kwetsbaarheid die wordt getest, waarbij client 1 de aanvaller is en client 2 het slachtoffer is. Er wordt in de figuur een veel gebruikte en simpele JavaScript functie genaamd alert gebruikt om de kwetsbaarheid van XSS aan te tonen. Wanneer het slachtoffer een pop-up in zijn/haar beeld krijgt, dan is er effectief sprake van een XSS kwetsbaarheid. XSS Kan opgedeeld worden in verschillende categorieën: reflected en stored XSS. Deze figuur is een voorbeeld van stored XSS want de code wordt opgeslagen in de database.

1.1.2 Reflected XSS

Figuur 2

Visualisatie reflected XSS bij zoek functie



Hepper (2017)

Bij reflected XSS wordt de code niet opgeslagen maar meteen gereflecteerd en teruggestuurd in de HTTP-response naar de browser van het slachtoffer. Een reflected XSS kwetsbaarheid komt vaak voor bij zoekfuncties, net zoals bij figuur 2. Er wordt dan JavaScript code meegegeven als zoekterm i.p.v. een realistische zoekterm en dit is exact wat er in figuur 2 gebeurt. De server in figuur 2 verwerkt het verzoek en geeft de zoek resultaten en de opgezochte term terug aan de gebruiker. Hierdoor bevindt de JavaScript zich in de HTTP-response. Doordat de code zich in de HTTP-response bevindt kan deze uitgevoerd worden door de browser van het slachtoffer en kan dit schade veroorzaken.

Een reflected XSS kwetsbaarheid stelt een aanvaller in staat om een kwaadaardige URL te maken met daarin JavaScript en deze door te sturen naar een slachtoffer. Het slachtoffer krijgt deze link en ziet dat deze naar de officiële (maar kwetsbare) webapp verwijst. Dit is gevaarlijk omdat de gebruiker er vanuit gaat dat alles in orde en veilig is omdat de link naar de officiële webapp gaat en dus is de kans groot dat het slachtoffer op de link klikt.

1.1.3 Stored XSS

Bij stored XSS wordt de JavaScript code opgeslagen op de server, vaak in een database. De code wordt dan automatisch uitgevoerd bij elke gebruiker die de opgeslagen content bekijkt of opvraagt. Deze kwetsbaarheid komt vaak voor bij gebruikers gegenereerde content zoals:

- Reacties, opmerkingen & reviews
- Forum & community berichten
- (contact)formulieren & rich text editors
- ...

Figuur 1 uit onderdeel 1.1 toont hoe het plaatsen van een opmerking kan misbruikt worden, als er een XSS kwetsbaarheid aanwezig is. De aanvaller plaatst een opmerking met kwetsbare JavaScript i.p.v. een realistische opmerking. Deze opmerking en code worden opgeslagen in de database van de webapp. Andere gebruikers laden de pagina met opmerkingen en met de kwetsbare JavaScript, deze JavaScript wordt vervolgens uitgevoerd in de browser van het slachtoffer. Dit is gevaarlijk omdat de gebruiker zelf niks moet doen, het gebeurt gewoon.

1.2 Hoe XSS voorkomen?

1.2.1 Algemeen

XSS kwetsbaarheden komen vooral voor door een fout of onvoorzichtigheid in de code van de frontend maar dit wilt niet zeggen dat er bij het opzetten van de backend en de API daar geen aandacht aan besteed moet worden. Specifiek voor stored XSS is het beter om te voorkomen dat er überhaupt uitvoerbare JavaScript code kan opgeslagen worden in de database. Om dit te voorkomen is het dus belangrijk dat alle gebruikersinput die de API moet verwerken eerst gecontroleerd en gevalideerd wordt of deze geen gevaarlijke (JavaScript) payload bevat.

1.2.2 HTML escaping - HTML entity encoding

Een manier om de gebruikersinput veilig te maken op vlak van XSS is door gebruik te maken van HTML escaping of een andere benaming HTML entity encoding. Bij HTML escaping worden alle of bepaalde karakters vervangen door hun vastgelegde HTML entities. Zodat de tekst niet geïnterpreteerd kan worden als HTML code door een browser, waardoor de code niet uitgevoerd wordt.

[De lijst](#) met karakters en hun HTML entities geeft aan welke entity naam of entity nummer bij dat karakter hoort. Deze lijst werd vastgelegd door de Web Hypertext Application Technology Working Group (WHATWG). Bij HTML escaping worden niet altijd alle karakters vervangen door hun HTML entities, vaak worden gewone letters behouden en enkel de speciale of invloedrijke karakters vervangen.

Hier is een voorbeeld van HTML escaping: het karakter kleiner dan < wordt vervangen door zijn entity naam **<** of door zijn entity nummer **<**; de gebruiker krijgt wel gewoon het kleiner dan teken te zien op zijn/haar beeldscherm. Een voorbeeld van een gevaarlijke XSS payload die verandert door middel van HTML escaping: de payload **** verandert dan in **** waardoor het niet langer als HTML code wordt geïnterpreteerd maar gewoon als tekst en niet wordt uitgevoerd.

1.2.3 HTML Sanitization & HTML Stripping

Een andere manier om de gebruikersinput veilig te maken op vlak van XSS is door gebruik te maken van HTML Sanitization of HTML Stripping. Bij HTML Sanitization en HTML Stripping worden bepaalde of alle HTML tags verwijderd uit een opgegeven string of tekst. Door het verwijderen van HTML tags kan de gebruikersinput geen code meer bevatten en dus geen schade meer aanbrengen. Een voorbeeld van HTML Stripping is dat de string bestaande uit **** gewoon volledig leeg wordt gemaakt omdat er geen tekst is buiten de HTML tag, er blijft een lege string over.

Het verschil tussen HTML Sanitization en HTML Stripping is dat bij sanitization worden enkel de niet tegenstaande HTML tags verwijderd, bij HTML Stripping worden alle HTML tags verwijderd uit een bepaalde tekst of string. Bij HTML Sanitization functies kan er dus een whitelist met toegestane HTML tags meegegeven worden, de tags in deze whitelist zullen dan niet verwijderd worden uit de tekst of string.

1.2.4 Cookies met HttpOnly attribuut

Als er wordt ingelogd met een account op een webapp wordt er vaak een sessie of access token aangemaakt en opgeslagen in de vorm van een cookie, deze is dan geldig voor een bepaalde tijdsperiode. Zodat de gebruiker tijdens deze tijdsperiode niet elke keer opnieuw zijn inloggegevens moet ingeven wanneer er een nieuwe API request gemaakt wordt. In plaats van de inloggegevens opnieuw mee te geven met de request wordt de cookie met de sessie of access token meegeven.

Een cookie kan bepaalde attributen en eigenschappen hebben die vertellen aan de browser hoe de cookie moet gebruikt worden. Zo bestaat er een attribuut genaamd HttpOnly, dit attribuut verteld aan de browser dat deze cookie niet uitgelezen mag worden via JavaScript. Als er toch geprobeerd wordt om deze cookie uit te lezen via JavaScript, dan doet de browser alsof deze niet bestaat. Dit beschermd de cookie met de opgeslagen sessie of access token tegen token theft en session hijacking via XSS, omdat de cookie onbereikbaar is via JavaScript.

Figuur 3

Uitlezen van een cookie zonder HttpOnly attribuut doormiddel van JavaScript

```
> document.cookie
< 'access_token_unsafe=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxIiwidXNlcm5hbWUiOiJJYW4iLCJyb2x1Ijo5LCJpYXQiOiE3NzE5NTIwNDIsImV4cCI6MTc3MTk1Mjk0Mn0.vZ8eVfT_V46aB_fUp2qs4QoK-VGp685e4iUe1R1J13g'
```

Zelfgemaakte afbeelding

Figuur 4

Uitlezen van een cookie met HttpOnly attribuut doormiddel van JavaScript

```
> document.cookie
< ''
```

Zelfgemaakte afbeelding

Figuur 3 en figuur 4 tonen het resultaat voor het opvragen van cookies met of zonder HttpOnly attribuut via JavaScript met de functie document.cookie. Deze functie werd uitgevoerd via de console van de DevTools in een Chromium browser.

1.3 Praktisch

Alle links naar het praktische deel van dit werk kunnen gevonden worden in de bijlagen van dit werk. Bij de webapps is er een pop-up systeem toegevoegd, deze geeft het succes of het mislukken van een request aan, dit zijn niet de JavaScript alert pop-ups. Figuur 7 toont hoe de JavaScript alert pop-ups (die aangeven dat XSS mogelijk is) eruit zien bij een Chromium browser.

1.3.1 Reflected XSS bij onveilige API

Figuur 5

Voorbeeld zoekfunctie met normale zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.

 × Search

Results for:

test

lan
23-2-2026, 22:31:10

Test
Dit is een test post om te kijken of alles werkt naar behoren.

Zelfgemaakte afbeelding

Het gebruikte praktische voorbeeld om reflected XSS aan te tonen voor dit onderzoek is een zoek functie waarmee een gebruiker kan zoeken naar bestaande posts, de gebruiker hoeft niet ingelogd te zijn. De zoekfunctie verwacht één of meerdere zoektermen en zal de zoekresultaten en de gebruikte zoekterm(en) terug geven aan de gebruiker. De zoek functie kan gebruikt worden via de webapp door onderaan de zoekbalk te gebruiken zoals in figuur 5 of door een query parameter te gebruiken in de URL, zoals hieronder:

<https://unsafe-app.ian-chains.be/?q=test>

Figuur 6

Voorbeeld zoekfunctie met XSS payload als zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.

 Search

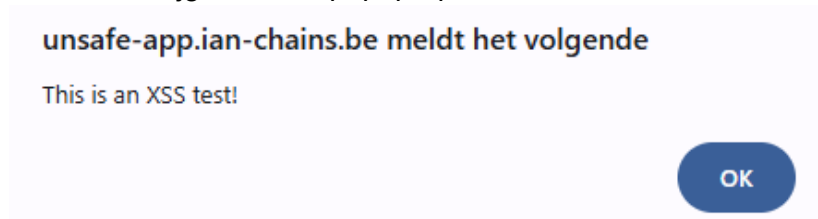
Results for:



Zelfgemaakte afbeelding

Figuur 7

Gebruiker krijgt JavaScript pop-up in beeld



Zelfgemaakte foto

Figuur 6 toont de zoekresultaten van een zoekopdracht met de XSS payload `<img src=x onerror=alert('This is an XSS test!')>` als zoekterm. Deze zoekopdracht kan uitgevoerd worden door de payload in te vullen in de zoekbalk of door volgende URL te gebruiken: [https://unsafe-app.ian-chains.be/?q=%3Cimg%20src=x%20onerror=%22alert\(%27This%20is%20an%20XSS%20test!%27\)%22%3E](https://unsafe-app.ian-chains.be/?q=%3Cimg%20src=x%20onerror=%22alert(%27This%20is%20an%20XSS%20test!%27)%22%3E) Als deze zoekopdracht wordt uitgevoerd blijkt dat de JavaScript code `alert('This is an XSS test!')` ook effectief uitgevoerd wordt en de gebruiker krijgt een pop-up zoals in figuur 7 te zien. Deze webapp is dus kwetsbaar voor XSS aanvallen. In figuur 6 is te zien dat de payload als HTML code herkent en uitgevoerd wordt, en er effectief geprobeerd wordt om de afbeelding in te laden maar doordat de afbeelding niet gevonden kan worden wordt het on error script uitgevoerd.

1.3.2 Stored XSS bij onveilige API

Figuur 8

Aanmaken van een post met normale content

Post Creation

Vul de velden in om een nieuwe post aan te maken.

Zelfgemaakte afbeelding

Figuur 9

Inladen van een post met normale content

Latest Posts

Bekijk hieronder de 5 laatst aangemaakte posts.

Ian
24-2-2026, 17:57:01

Welkom!
Welkom, dit is mijn eerste post op deze site!

Zelfgemaakte afbeelding

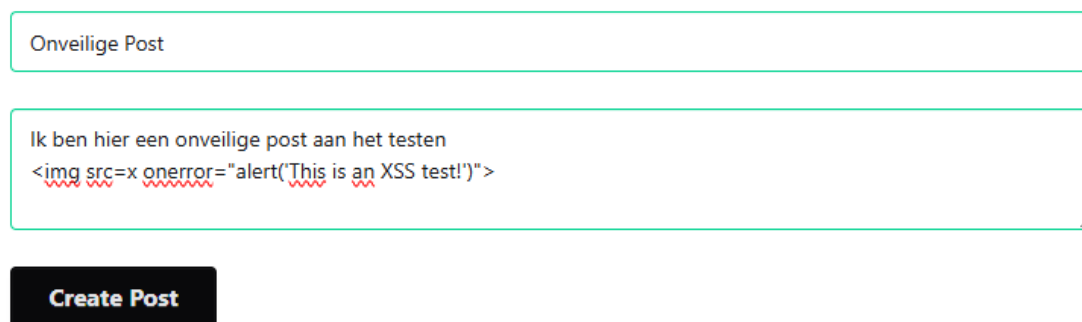
Het gebruikte praktische voorbeeld om stored XSS aan te tonen voor dit onderzoek is het kunnen aanmaken en opvragen van posts. Voor het aanmaken van posts moet een gebruiker ingelogd zijn met een account. Voor het inladen van posts worden telkens de 5 laatst aangemaakte posts weergegeven door te klikken op de load button. Figuur 8 toont hoe het aanmaken van een nieuwe post met normale content er uit ziet. Figuur 9 toont hoe deze post met normale content eruit ziet bij het inladen.

Figuur 10

Aanmaken van post met XSS payload als content

Post Creation

Vul de velden in om een nieuwe post aan te maken.



Onveilige Post

Ik ben hier een onveilige post aan het testen

Create Post

Zelfgemaakte afbeelding

Figuur 11

Inladen van post met XSS payload als content

Latest Posts

Bekijk hieronder de 5 laatst aangemaakte posts.



Ian
24-2-2026, 18:02:20

Onveilige Post

Ik ben hier een onveilige post aan het testen

Zelfgemaakte afbeelding

Figuur 10 toont het aanmaken van een nieuwe post met zowel normale content als dezelfde XSS payload uit onderdeel 1.3.1. Figuur 11 toont hoe deze nieuwe post met XSS payload eruit ziet bij het inladen, de normale content wordt op een normale manier afgebeeld maar de XSS payload wordt weer als HTML code geïnterpreteerd. Dezelfde JavaScript code wordt weer opnieuw uitgevoerd en de gebruiker krijgt opnieuw dezelfde pop-up te zien zoals in figuur 7.

1.3.3 Veilige API, beschermt tegen XSS

Om de API en het volledige webapp systeem te beschermen van XSS is er in dit onderzoek voor gekozen om HTML escaping of HTML entity encoding te gebruiken door middel van de escape functie van de [MarkupSafe](#) module in Python. Bij deze veilige API wordt er eerst HTML escaping uitgevoerd op alle gebruikersinput voordat het verder verwerkt wordt door het systeem.

Figuur 12

Voorbeeld veilige zoekfunctie met XSS payload als zoekterm

Post Search

Vul hieronder een zoekterm in om posts te zoeken die deze term bevatten.

Q Search

Results for:

Zelfgemaakte afbeelding

Figuur 13

Gebruikersinput waar HTML escaping op werd uitgevoerd, foto uit DevTools

```
▼ {detail: "No posts found",...}  
  detail: "No posts found"  
  query: "&lt;img src=x onerror= &#34;alert(&#39;This is an XSS test!&#39;)&#34;&gt;";
```

Zelfgemaakte afbeelding

Figuur 12 toont de veilige zoekopdracht van dezelfde XSS payload als uit onderdeel 1.3.1. Bij het uitvoeren van deze zoekopdracht krijgt de gebruiker geen Pop-up in beeld, dus de XSS payload heeft niet gewerkt. Het verschil is duidelijk zichtbaar in de manier waarop de gebruikte zoekterm wordt afgebeeld, de zoekterm wordt niet meer afgebeeld als HTML code. Het is niet meer de foto die niet gevonden kan worden en een error oplevert, maar de gebruikte zoekterm wordt gewoon als tekst afgebeeld. Dit komt doordat de API de escaped versie van de gebruikte zoekterm terug geeft aan de gebruiker waardoor het niet meer als HTML code kan geïnterpreteerd worden.

Voor het aanmaken en inladen van een post gebeurt net hetzelfde, de content van de post wordt eerst geëscaped voordat het wordt opgeslagen in de database, daardoor kan er geen enkele XSS payload opgeslagen worden in de database.

De cookie met de access token kreeg de extra attribuut HttpOnly toegewezen zodat de cookie niet meer uit leesbaar is via JavaScript, om op deze manier te beschermen tegen token theft via XSS.

2 CSRF - Cross-Site Request Forgery

2.1 Wat is CSRF?

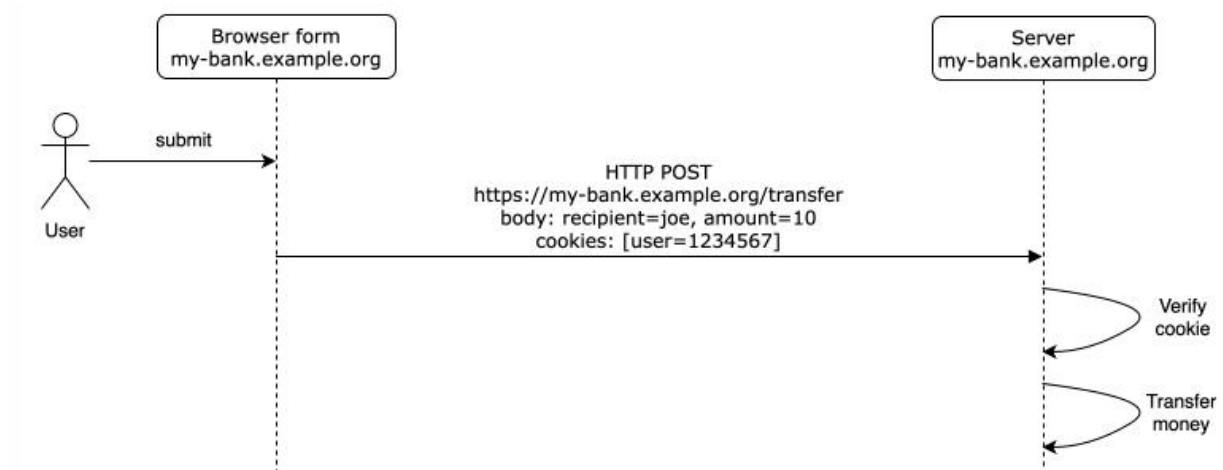
2.1.1 Algemeen

CSRF is een beveiligingsprobleem waarbij het mogelijk is om ongewenste HTTP requests te versturen vanaf een andere site (cross-site) zonder dat de gebruiker dit wilde en meestal zonder enige actie uit te voeren. Door deze requests kan er data gedeeld of aangepast worden vanaf een andere site, zonder dat de gebruiker dit wilde en kan een request een kwaadaardig effect hebben.

2.1.2 Voorbeeld Normaal Request

Figuur FXXFF

Voorbeeld HTTP POST request, geld overschrijven



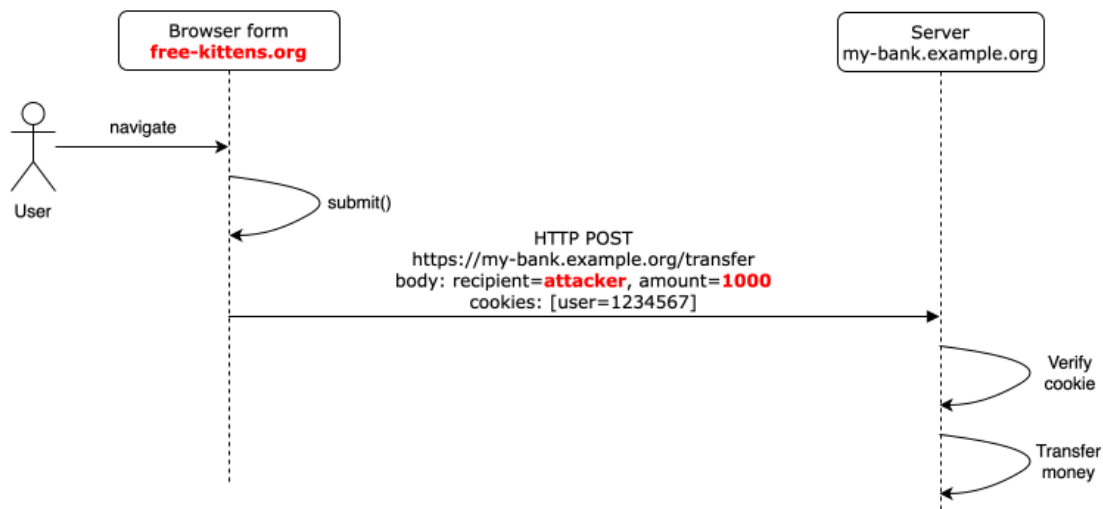
(Cross-site Request Forgery (CSRF) - Security | MDN, 2025)

Figuur FXXFF toont een POST request van een API waarbij de gebruiker zich op de webapp `my-bank.example.org` bevindt en geld wil overschrijven. De gebruiker voert de actie uit dus wordt er een HTTP POST request verstuurd. De gebruiker is reeds ingelogd en heeft dus een user UID of authenticatie token in de cookies van zijn browser opgeslagen. Deze cookies worden automatisch door de browser altijd meegeven met een HTTP request. De server ontvangt deze request en controleert de user UID en/of de authenticatie token, als deze tokens geldig zijn dan wordt de request verwerkt en het geld overgeschreven.

2.1.3 Voorbeeld CSRF

Figuur FXXFF

Voorbeeld CSRF bij POST request van figuur FXXFF



(Cross-site Request Forgery (CSRF) - Security | MDN, 2025)

Figuur FXXFF toont dezelfde POST request als figuur FXXFF maar de gebruiker bevindt zich deze keer op een kwaadaardige webapp. Deze webapp zal zelf zonder enige aanleiding of actie een HTTP POST request versturen in naam van de gebruiker om geld over te schrijven. De browser herkent de hostname van de request en linkt deze aan de cookies van de bank webapp en geeft deze cookies en authenticatie tokens automatisch mee in de request. De server ontvangt deze request en zal deze verwerken als de authenticatie tokens nog geldig zijn. Een CSRF-aanval maakt dus gebruik van het feit dat een gebruiker reeds ingelogd is en authenticatie tokens reeds opgeslagen zijn als cookie, die worden dan automatisch meegegeven door de browser.

2.2 Hoe CSRF voorkomen?

2.2.1 SOP & CORS

Moderne browsers maken gebruik van de Same-Origin Policy (SOP) beveiligingsmaatregel om CSRF te voorkomen. SOP is een maatregel aan de clientside en zorgt ervoor dat er enkel gevoelige data tussen webapps kan uitgewisseld worden als deze same-origin³ zijn. Als er niet wordt voldaan aan de SOP dan wordt de data van de API request geblokkeerd en kan de andere webapp deze data niet gebruiken/lezen. Zodat een andere (kwaadaardige) webapp die cross-origin⁴ is, geen gevoelige data kan lezen van een andere webapp.

Cross-Origin Resource Sharing (CORS) is een uitzondering op SOP, met CORS kan een andere webapp toegang krijgen om data te gebruiken van een andere API als deze cross-origin zijn.

PREFLIGHTS UITLEGGEN WAT IS EEN SIMPLE REQUEST EN WAT NIET
POST UITLEGGEN DAT HET EEN SIMPLE IS EN TOCH UITGEVOERD WORDT MAAR
DATA NIET DOOR FRONTEND KAN VERWERKT WORDEN

2.2.2 Server Side Origin Check

Niet altijd het veiligste CSRF tokens worden verkozen bovenop dit

2.2.3 CSRF Tokens

Signed tokens

2.3 Praktisch

X

Cookies met samesite attriboot eerst uitleggen voor CSRF tokens

³ Meerdere webapps of pagina's zijn **same-origin** aan elkaar als ze dezelfde protocol (HTTPS), hostname en poort hebben.

⁴ **Cross-Origin** is het tegenovergestelde van same-origin. Wanneer 1 van de eigenschappen niet volledig overeenkomt, wordt dit aangeduid als cross-origin.

Besluit

- **Nog aanpassen en ook toevoegen van CSRF, dus controleren of de request wel echt is en niet zonder toestemming is gemaakt**

Het is duidelijk dat er één grote lijn en overeenkomst aanwezig is tussen al deze verschillende veelvoorkomende beveiligingsproblemen bij REST-API's. De directe input, gegevens en acties van de gebruiker of client kan nooit vertrouwd worden en moet altijd eerst gecontroleerd en opgekuist worden voordat deze input en gegevens verwerkt worden. Zodat de input geen schadelijke payload of code bevat en geen schade kan aanbrengen aan het digitale systeem, de server, de databank of andere gebruikers. Dus er moet altijd eerst gepoetst worden voordat de input kan vertrouwd worden, eens dat is gebeurd kan de input verwerkt worden.

Deze input gegevens kunnen gepoetst worden op verschillende manieren namelijk:

Bijlagen

Voor het praktische gedeelte van dit onderzoek is er gebruik gemaakt van een GitHub repository om de bestanden, de wijzigingen en documentatie overzichtelijk te kunnen bijhouden.

Alle bijlagen werden samen gebundeld tot een zip bestand. Het zip bestand werd samen met dit werk ingediend. Je kan de bijlagen ook downloaden via volgende link:

Info & Onderwerp:	Bestandsnaam of link:
Referentielijst van dit onderzoek	25-26_GDC_Referentielijst_Ian_6ICWE.pdf
GitHub Repository	https://github.com/GO-atheneum-De-Tandem/GDC-Ian-2025-2026
Onveilig API Systeem	https://unsafe-api.ian-chains.be
Onveilig API Systeem – OpenAPI Docs	https://unsafe-api.ian-chains.be/docs
Onveilige Webapp (Frontend)	https://unsafe-app.ian-chains.be
Veilig API Systeem	https://safe-api.ian-chains.be
Veilig API Systeem – OpenAPI Docs	https://safe-api.ian-chains.be/docs
Veilige Webapp (Frontend)	https://safe-app.ian-chains.be
CSRF Attacker Webapp op Onveilige API	https://attacker-unsafe.yourwebmaster.be
CSRF Attacker Webapp op Veilige API	https://attacker-safe.yourwebmaster.be

Referentielijst

De referentielijst is opgenomen in een apart bestand om dit onderzoek overzichtelijk te houden. De referentielijst is ook gesorteerd per onderdeel van dit werk. Dit bestand kan je terug vinden in de meegeleverde bijlagen.

het controleren van spelling van dit werkstuk is er gebruik gemaakt van woordenlijst.org en spelling.nu.